VANDERBILT UNIVERSITY

Neural ODEs

Dynamic Surrogate Modeling

Blaine Swieder

CS 8395-04



What Is a Neural ODE?

A neural network represents a differential equation’s right-hand side.

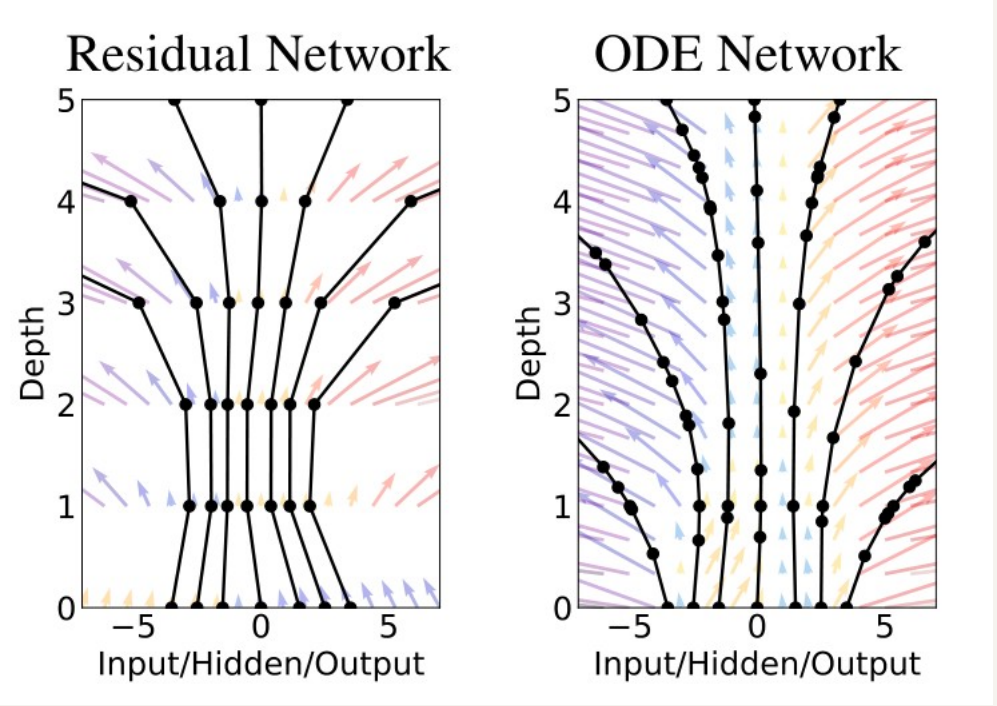
$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = f_\theta(t, \mathbf{x}(t)) \,, \qquad \mathbf{x}(t_0) = \mathbf{x}_0$$

- The state x describes the system at time t.
- The network fθ predicts how that state changes.
- The initial state and an ODE solver determine the trajectory.
- In physical models t is time. In feature models it can represent network depth.

Continuous Depth and Residual Networks

$$\mathbf{h}_{k+1} = \mathbf{h}_k + \Delta t\, f_\theta(t_k, \mathbf{h}_k) \quad \longleftrightarrow \quad \frac{\mathrm{d}\mathbf{h}}{\mathrm{d}t} = f_\theta(t, \mathbf{h})$$

- A residual update resembles an Euler step with step size Δt.
- A Neural ODE specifies a vector field instead of a fixed number of layers.
- The solver chooses the evaluation points needed for integration.



Original comparison: Chen et al. (2018), Figure 1

How a Neural ODE Learns

$$\mathbf{x}(t) = \mathbf{x}_0 + \int_{t_0}^{t} f_\theta(\tau, \mathbf{x}(\tau))\ \mathrm{d}\tau$$

- Integrate the learned derivative from a known initial state.
- Compare predicted and observed states at measurement times.
- Differentiate the trajectory loss with respect to network parameters.
- Update the network and repeat using training trajectories.

Training can differentiate solver operations directly or use an adjoint sensitivity system.

Connection to Surrogate Modeling

Surrogate	Learned Relation	Predicted Quantity
Static	Parameters to response	Steady deflection or temperature
Dynamic	State to time derivative	Trajectory after numerical integration

$$\dot{\mathbf{x}} = f_\theta(t, \mathbf{x}, \mathbf{u}, \mathbf{p})$$

Controls u and physical parameters p require corresponding inputs and training coverage.

Paper 1

Neural Ordinary Differential Equations

Chen, Rubanova, Bettencourt, and Duvenaud

NeurIPS 2018

Paper 1: Overview

Chen, Rubanova, Bettencourt, and Duvenaud · NeurIPS 2018

Research question: How can we train a neural network through an ODE solution?

- Continuous-depth residual architectures.
- Adjoint sensitivities for end-to-end training through ODE solutions.
- Latent ODEs for time series with irregular observations.
- Continuous normalizing flows for density modeling.

Paper 1: Adjoint Sensitivity Method

$$\mathbf{a}(t) = \nabla_{\mathbf{z}(t)}\mathcal{L}, \qquad \dot{\mathbf{a}}(t) = -\left(\frac{\partial f_\theta}{\partial \mathbf{z}}\right)^\top \mathbf{a}(t)$$

- Solve the state equation forward, then propagate loss sensitivities backward.
- Automatic differentiation supplies vector–Jacobian products.
- Intermediate observation losses introduce adjoint updates at observation times.
- Memory savings concern solver depth. Reconstruction and numerical error remain.

Paper 1: Model Families and Experiments

Model Family	Method	Evaluation
ODE-Net classifier	Replace residual blocks with an ODE module	MNIST test error and parameter count
Latent ODE	Infer an initial latent state, integrate, then decode	Noisy spirals with irregular samples
Continuous normalizing flow	Integrate state and log density together	Density-estimation experiments

$$\frac{\mathrm{d}\log p(\mathbf{z}(t))}{\mathrm{d}t} = -\operatorname{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{z}}\right)$$

The divergence of the vector field determines the continuous change in log density.

Paper 1: Classification Results

Reported MNIST Experiment

Model	Test Error	Parameters	Memory in Depth
ResNet	0.41%	0.60 million	O(L)
RK-Net	0.47%	0.22 million	O(L̃)
ODE-Net	0.42%	0.22 million	O(1)

ODE-Net achieved similar test error with fewer parameters in this experiment.

These results do not establish a universal wall-clock speed advantage. Source: Chen et al., Table 1.

Paper 1: Irregular Time-Series Results

Reported Predictive RMSE for Noisy Spiral Trajectories

Observed Points	RNN	Latent ODE
30 / 100	0.3937	0.1642
50 / 100	0.3202	0.1502
100 / 100	0.1813	0.1346

Latent ODE reduced predictive error across the reported sampling levels.

Extrapolation capability does not guarantee reliable long-horizon forecasts. Source: Chen et al., Table 2.

Paper 1: Five Takeaways

- A neural network can learn the derivative of a continuous state.
- A numerical ODE solver becomes part of the model’s computation.
- Adjoint sensitivities provide a memory-efficient training route.
- Latent dynamics support irregular observations.
- Continuous flows link differential equations with density modeling.

Paper 2

Augmented Neural ODEs

Dupont, Doucet, and Teh

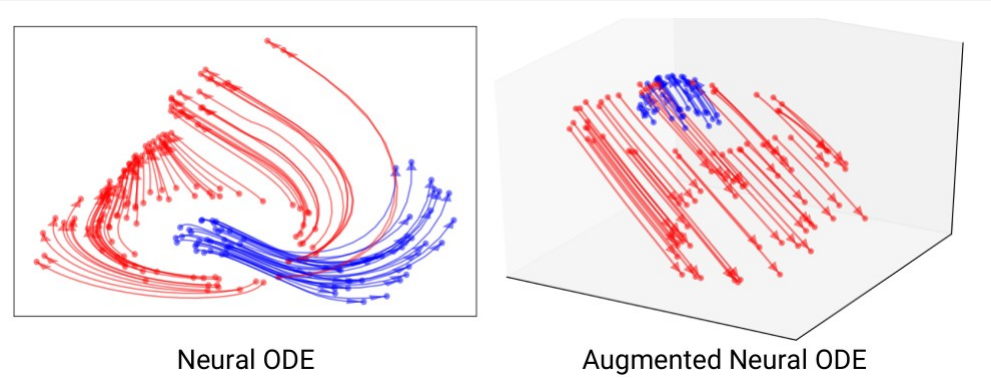
NeurIPS 2019

Paper 2: Overview

Dupont, Doucet, and Teh · NeurIPS 2019

Research question: Which flow restrictions can added state coordinates address?

- Under regularity assumptions, a fixed-time flow preserves topology.
- Distinct full states cannot merge at a common time.
- Difficult transformations can require complicated, expensive flows.



The restriction concerns the flow and its architecture. Projections and readouts can change what is observed.

Paper 2: Augmentation Method

$$\mathbf{y}(t) = \begin{bmatrix} \mathbf{h}(t) \\ \mathbf{a}(t) \end{bmatrix}, \qquad \mathbf{y}(t_0) = \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{0} \end{bmatrix}, \qquad \dot{\mathbf{y}} = F_\theta(t, \mathbf{y})$$

- Append extra coordinates, initialized at zero, to the original state.
- Learn dynamics in the expanded state space.
- Apply a projection or readout to obtain the desired output.
- Full-state uniqueness remains. Projected trajectories have more flexibility.

Paper 2: Reported Image Results

Test Accuracy: Mean ± Standard Deviation over Five Runs

Dataset	NODE	ANODE
MNIST	96.4% ± 0.5	98.2% ± 0.1
CIFAR10	53.7% ± 0.2	60.6% ± 0.4
SVHN	81.0% ± 0.6	83.5% ± 0.5

The authors also report lower losses and fewer function evaluations with augmentation.

Within-paper comparison. These architectures differ from Chen et al.’s MNIST classifiers.

Paper 2: Five Takeaways

- Unique flows impose restrictions on representable transformations.
- Adding state coordinates can remove some restrictions after observation.
- Simpler learned flows can reduce numerical evaluation cost.
- Reported benefits include fitting and generalization under matched comparisons.
- Augmentation needs tuning and remained slower than ResNets in this paper.

Demonstration

Silverbox

Neural ODEs with Measured Oscillator Data

Wigren and Schoukens Dataset

Demo: Measured Nonlinear Dynamics

Demo question: Can a learned ODE forecast measured voltage under a new input?

- Silverbox is an electronic circuit with Duffing-like oscillator behavior.
- The dataset records the applied input voltage and the output voltage.
- The model predicts the output using a numerical internal state and the known input.
- The comparison includes a linear model, a Neural ODE, and an augmented Neural ODE.

Demo: Dataset and Evaluation Protocol

Record	Samples	Raw Source Indices	Purpose
Training	8,192	40,986 to 49,177	Fit normalization and models
Validation	2,048	58,370 to 60,417	Select checkpoints
Multisine test	2,048	105,712 to 107,759	Independent forecast
Arrow test clip	2,048	35,000 to 37,047	Larger-amplitude response

610.35 Hz. Each test clip uses 50 initial output samples, then scores 1,998 input-only predictions.

Demo: Internal State and Learned Dynamics

$$\frac{\mathrm{d}\mathbf{z}}{\mathrm{d}\tau} = A\mathbf{z} + B\widetilde{u}(\tau) + g_\theta([\mathbf{z}, \widetilde{u}]), \qquad \widehat{y} = \mu_y + \sigma_y z_1$$

- A and B come from a training-only linear warm start.
- The network gθ learns a nonlinear correction to that vector field.
- Two coordinates initialize from the last two normalized outputs in the prefix.
- The augmented model adds a third coordinate initialized at zero.
- A projection onto the first coordinate predicts the measured output.

The state coordinates are numerical representation variables. Voltage alone is not assumed to be a complete state.

Demo: Code Overview

Component	Implementation
Measured data	Four local CSV files with source indices and checksums
Model	InputDrivenODE with two or three coordinates
Nonlinear correction	Two tanh hidden layers, width 16
Integration	torchdiffeq.odeint, fixed-step RK4
Training and selection	Adam trajectory loss, validation-selected checkpoint
Execution	CPU worker, fixed seed, 90 s execution limit

The notebook runs silverbox_demo.py. config.json fixes the shared experiment settings.

Demo: Training through the Solver

$$\mathcal{L}(\theta) = \frac{1}{BN} \sum_{b=1}^{B} \sum_{i=1}^{N} \left(\frac{\widehat{y}_\theta^{(b)}(t_i) - y^{(b)}(t_i)}{\sigma_y} \right)^2$$

B = 16 windows, N = 25 points. Each model receives 120 Adam updates.

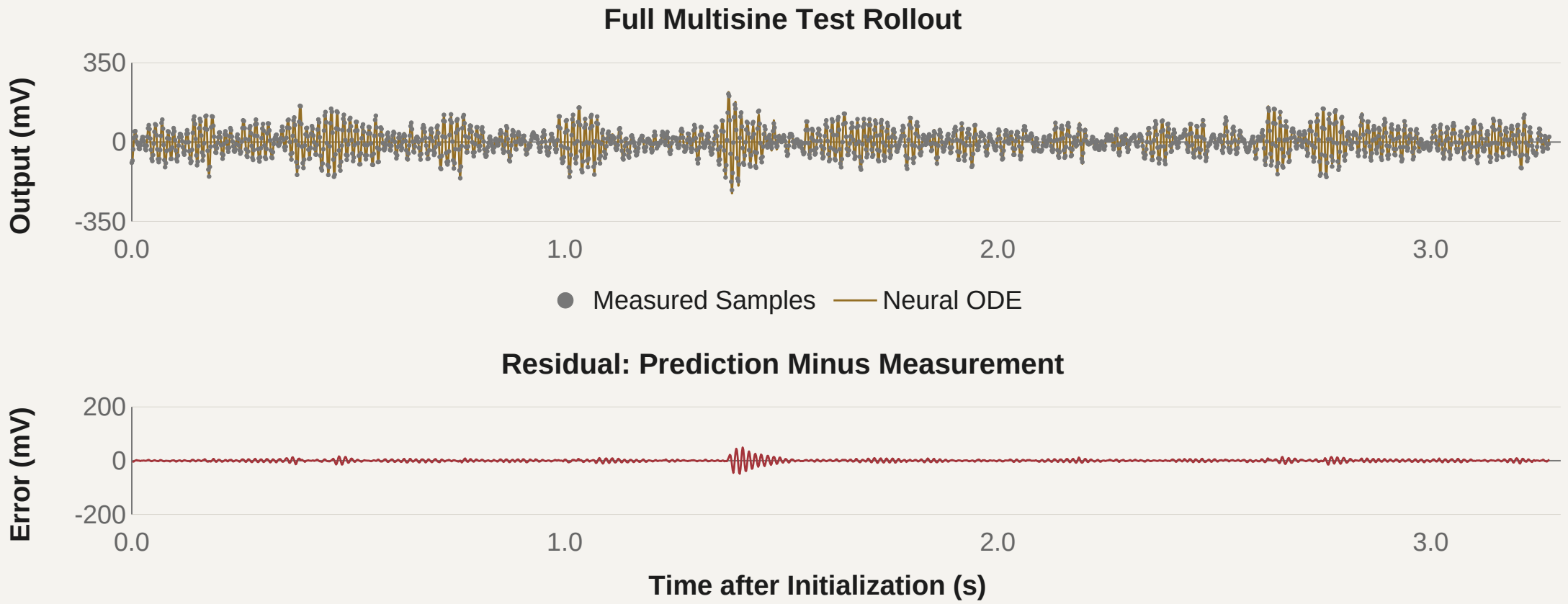
The code below uses normalized targets and predictions.

```
optimizer.zero_grad(set_to_none=True)
prediction = odeint(model, state, times,
    method='rk4', options={'step_size': 0.5})[..., 0]
loss = (prediction - target).square().mean()
loss.backward()
clip_grad_norm_(model.parameters(), 10.0)
optimizer.step()
```

Training and evaluation use the same integration step. Input voltage is linearly interpolated between samples.

Demo Results: Full Test Response and Error

Full RMSE: 5.706 mV Maximum |Error|: 49.250 mV



All 1,998 scored errors are shown. Residual = prediction minus measurement.

Demo Results: Output Prediction Errors

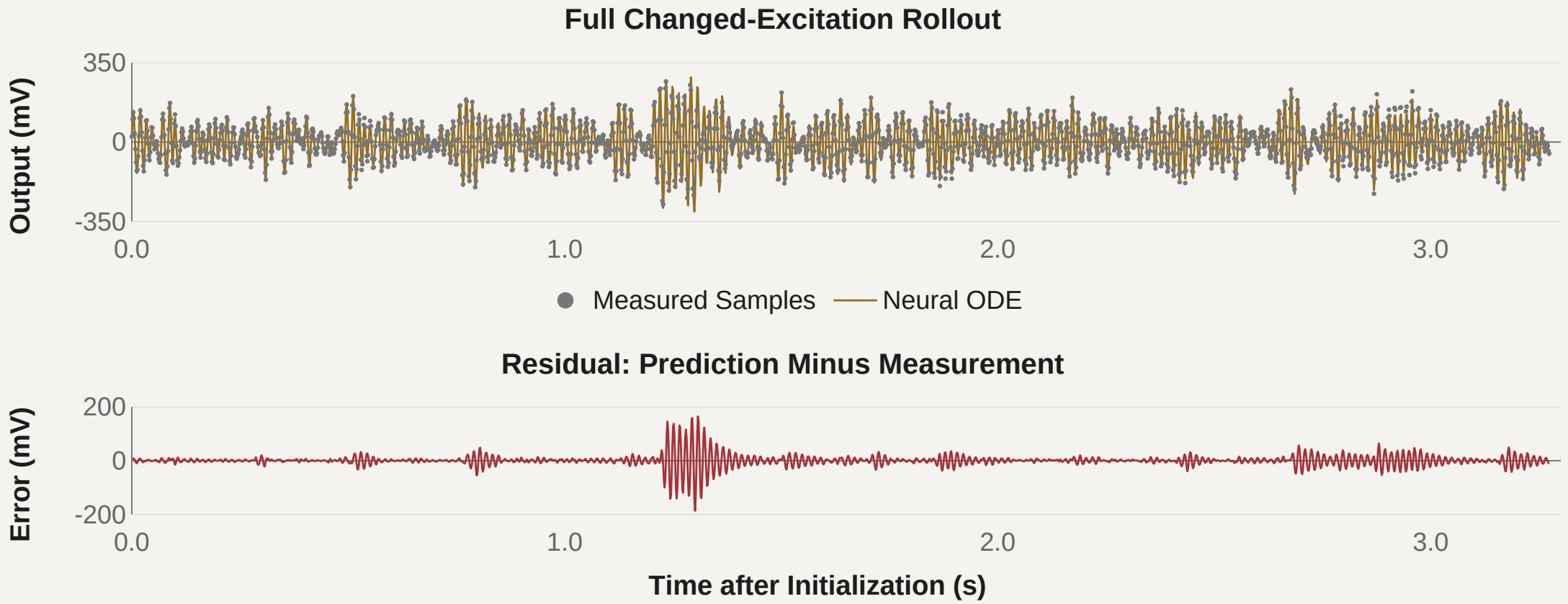
$$\mathrm{RMSE}_{\mathrm{mV}} = 1000\sqrt{\frac{1}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)^2}$$

Model	Multisine RMSE (mV)	Arrow-Clip RMSE (mV)
Linear ODE	18.060	27.454
Neural ODE	5.706	22.566
Augmented Neural ODE	4.435	20.834

Both neural models improve on the linear baseline in this run. Errors increase on the larger-amplitude clip.

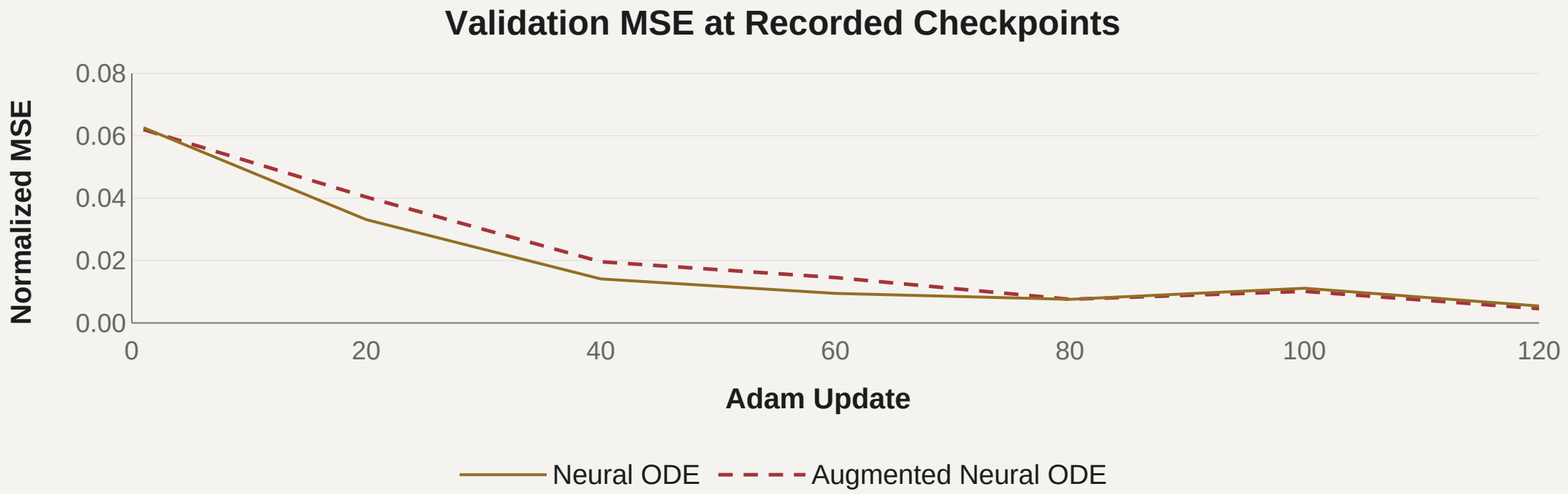
Demo Results: Changed Excitation and Error

Full RMSE: 22.566 mV Maximum |Error|: 186.707 mV



All 1,998 scored errors are shown. Input amplitude and waveform both change.

Demo: Augmentation and Validation



Test RMSE decreases by 22.3% with one added coordinate in this run.

Neural ODE: 370 neural parameters. Augmented: 403. One seed does not establish a general advantage.

Demo: Numerical Checks and Reproducibility

$$\tau = \frac{t - t_0}{\Delta t}, \qquad \Delta t = \frac{1}{610.35}\,\mathrm{s}, \qquad h = \frac{\Delta t}{2} \approx 0.819\,\mathrm{ms}$$

- Halving the RK4 step changes validation output by at most 0.298 mV.
- The same check changes test output by at most 0.343 mV.
- Both fresh runs reproduce the same weights and prediction CSV files on the tested stack.
- Seed 17, CPU float64, one thread, local CSV data, and a fixed configuration define the run.

The step check measures numerical sensitivity. It does not give an exact continuous-time error bound.

How the Papers Connect to the Demo

Source Concept	Implementation in This Demo
Chen: learned ODE dynamics	A network supplies a derivative and a solver produces the response
Chen: training through integration	Direct autograd through RK4; continuous-adjoint memory savings are not claimed
Dupont: state augmentation	Append a zero-initialized coordinate, integrate jointly, project to output

The two papers establish the methods. Silverbox supplies a separate measured application dataset.

Advantages of the Approach

- Continuous-time predictions under a specified input trajectory.
- End-to-end training through the response calculation.
- A neural correction can extend a simpler identified model.
- Latent and augmented coordinates can represent unobserved dynamics.
- A reusable dynamic surrogate within the coverage checked by validation.

Limitations: Model and Data

- Unmeasured internal states require an initialization or inference rule.
- Errors increase on a clip with changed amplitude and waveform.
- Voltage predictions do not identify unique physical oscillator parameters.
- One seed and short test clips do not establish full-benchmark performance.
- Augmentation changes parameter count and does not guarantee improvement.

Limitations: Numerical Computation

- A finer step raises the number of network evaluations.
- Model error and numerical integration error both affect forecasts.
- Stiff learned dynamics can force expensive numerical solves.
- Continuous-adjoint gradients depend on reconstruction and solver choices.
- Physical stability and reliable long-horizon extrapolation require separate checks.

Conclusion

- Neural ODEs learn a vector field that generates a dynamic response.
- Chen et al. establish trainable continuous-depth models.
- Dupont et al. explain flow restrictions and introduce state augmentation.
- Silverbox connects those ideas to measured nonlinear input-output behavior.
- The demo shows lower test error than a linear baseline and larger errors under changed excitation.

Questions and Discussion

Questions about the methods or the demo?

- How should we choose a state when only input and output are measured?
- What evidence would justify adding more latent coordinates?
- How far beyond the observed excitation would you trust this surrogate?

Sources: Research Papers

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D.
Neural Ordinary Differential Equations
Advances in Neural Information Processing Systems 31 (2018)

https://arxiv.org/abs/1806.07366

Dupont, E., Doucet, A., and Teh, Y. W.
Augmented Neural ODEs
Advances in Neural Information Processing Systems 32 (2019)

https://arxiv.org/abs/1904.01681

Sources: Data, Software, and Evaluation

Wigren and Schoukens

Three Free Data Sets for Development and Benchmarking in Nonlinear System Identification
ECC, pp. 2933–2938 (2013)
DOI: 10.23919/ECC.2013.6669201

Silverbox Data and Protocol

https://www.nonlinearbenchmark.org/benchmarks/silverbox

torchdiffeq

https://github.com/rtqichen/torchdiffeq

Vanderbilt PowerPoint Template

https://www.vanderbilt.edu/refd/boc/

Schoukens: Simulation-Error Evaluation

Improved Initialization of State-Space Artificial Neural Networks
2021 European Control Conference, 1913–1918
Equations (6), (8), and Figure 5

DOI: 10.23919/ECC54610.2021.9655207

Model–Data Comparison and Residuals

https://www.mathworks.com/help/ident/ref/compare.html